

Minimize Cost by Branch and Bound

Zhesheng Xu

Student ID: 42353012

May 25th 2025

1 题目描述

1.1 问题背景

任务分配问题 (Assignment Problem) 是运筹学中的经典问题。给定 n 个人和 n 个工作，每个人完成每个工作都有相应的成本，要求找到一个一对一的分配方案，使得总成本最小。

1.2 具体问题

给定 4 个人 (A、B、C、D) 和 4 个工作 (Job1、Job2、Job3、Job4)，成本矩阵如下：

人员	Job1	Job2	Job3	Job4
A	9	2	7	8
B	6	4	3	7
C	5	8	1	8
D	7	6	9	4

目标：找到最小总成本的分配方案。

约束条件：

- 每个人必须分配到一个工作
- 每个工作必须分配给一个人
- 一对一分配关系

2 算法分析

2.1 分支限界算法原理

分支限界 (Branch and Bound) 算法是一种求解组合优化问题的精确算法，其基本思想是：

1. **分支 (Branching)**：将原问题分解为若干子问题
2. **限界 (Bounding)**：对每个子问题计算上界或下界
3. **剪枝 (Pruning)**：利用界限信息删除不可能产生最优解的分支

2.2 算法设计要素

2.2.1 状态空间表示

- **状态**：部分分配方案，记录已分配的人员-工作对
- **初始状态**：空分配
- **目标状态**：完全分配

2.2.2 分支策略

按人员顺序进行分支，为每个人尝试分配所有未被占用的工作。

2.2.3 限界函数

对于当前节点，计算下界：

$$LB = \text{当前成本} + \sum_{i \in \text{未分配人员}} \min_{j \in \text{未分配工作}} c_{ij}$$

其中 c_{ij} 表示人员 i 完成工作 j 的成本。

2.2.4 剪枝条件

如果某节点的下界 $LB \geq UB$ (当前最优解)，则剪枝该分支。

3 算法伪代码

Algorithm 1 分支限界算法解决任务分配问题

Require: 成本矩阵 $C[n][n]$

Ensure: 最优分配方案和最小成本

```
1: 初始化:  $UB \leftarrow +\infty$ ,  $bestSolution \leftarrow \emptyset$ 
2: 创建根节点:  $assignment \leftarrow \emptyset$ ,  $cost \leftarrow 0$ ,  $level \leftarrow 0$ 
3: 计算根节点下界:  $LB \leftarrow calculateLowerBound(\emptyset)$ 
4: 将根节点加入优先队列  $Q$ 
5:
6: while  $Q$  非空 do
7:   从  $Q$  中取出下界最小的节点  $node$ 
8:   if  $node.LB \geq UB$  then
9:     剪枝, 继续下一次循环
10:  end if
11:
12:  if  $node.level = n$  then
13:    {完整解}if  $node.cost < UB$  then
14:       $UB \leftarrow node.cost$ 
15:       $bestSolution \leftarrow node.assignment$ 
16:    end if
17:  else
18:     $person \leftarrow node.level$ 
19:    for 每个未分配工作  $job$  do
20:      创建新子节点  $child$ 
21:       $child.assignment \leftarrow node.assignment \cup \{(person, job)\}$ 
22:       $child.cost \leftarrow node.cost + C[person][job]$ 
23:       $child.level \leftarrow node.level + 1$ 
24:       $child.LB \leftarrow calculateLowerBound(child.assignment)$ 
25:
26:      if  $child.LB < UB$  then
27:        将  $child$  加入队列  $Q$ 
28:      end if
29:    end for
30:  end if
31: end while
32:
33: return  $bestSolution$ ,  $UB$ 
```

Algorithm 2 计算下界函数

Require: 当前分配 *assignment*, 成本矩阵 *C*

Ensure: 下界值 *LB*

```
1: currentCost  $\leftarrow \sum_{(i,j) \in \text{assignment}} C[i][j]$ 
2: unassignedPersons  $\leftarrow \{0, 1, \dots, n-1\} - \{i : (i, j) \in \text{assignment}\}$ 
3: unassignedJobs  $\leftarrow \{0, 1, \dots, n-1\} - \{j : (i, j) \in \text{assignment}\}$ 
4:
5: lowerBound  $\leftarrow \text{currentCost}$ 
6: for 每个未分配人员 person do
7:   minCost  $\leftarrow \min_{\text{job} \in \text{unassignedJobs}} C[\text{person}][\text{job}]$ 
8:   lowerBound  $\leftarrow \text{lowerBound} + \text{minCost}$ 
9: end for
10:
11: return lowerBound
```

4 Python 实现

Listing 1: 分支限界算法 Python 实现

```
1 import heapq
2 from copy import deepcopy
3
4 class Node:
5     def __init__(self, assignment=None, cost=0, level=0, lower_bound=0):
6         self.assignment = assignment if assignment else {}
7         self.cost = cost
8         self.level = level
9         self.lower_bound = lower_bound
10
11     def __lt__(self, other):
12         return self.lower_bound < other.lower_bound
13
14 def calculate_lower_bound(cost_matrix, assignment, n):
15     """ 计算当前节点的下界 """
16     current_cost = sum(cost_matrix[person][job]
17                        for person, job in assignment.items())
18
19     assigned_persons = set(assignment.keys())
20     assigned_jobs = set(assignment.values())
```

```

21
22     unassigned_persons = [i for i in range(n) if i not in
23         assigned_persons]
24
25     unassigned_jobs = [j for j in range(n) if j not in assigned_jobs]
26
27     lower_bound = current_cost
28
29     for person in unassigned_persons:
30         min_cost = min(cost_matrix[person][job] for job in
31             unassigned_jobs)
32         lower_bound += min_cost
33
34     return lower_bound
35
36 def branch_and_bound_assignment(cost_matrix):
37     """分支限界算法解决任务分配问题"""
38     n = len(cost_matrix)
39
40     # 初始化
41     upper_bound = float('inf')
42     best_solution = None
43
44     # 创建根节点
45     root = Node()
46     root.lower_bound = calculate_lower_bound(cost_matrix, {}, n)
47
48     # 优先队列（最小堆）
49     queue = [root]
50     heapq.heapify(queue)
51
52     nodes_explored = 0
53     nodes_pruned = 0
54
55     while queue:
56         current_node = heapq.heappop(queue)
57         nodes_explored += 1
58
59         # 剪枝
60         if current_node.lower_bound >= upper_bound:
61             nodes_pruned += 1
62             continue

```

```

59
60     # 检查是否为完整解
61     if current_node.level == n:
62         if current_node.cost < upper_bound:
63             upper_bound = current_node.cost
64             best_solution = deepcopy(current_node.assignment)
65             continue
66
67     # 分支：为当前人员分配工作
68     person = current_node.level
69     assigned_jobs = set(current_node.assignment.values())
70
71     for job in range(n):
72         if job not in assigned_jobs:
73             # 创建子节点
74             new_assignment = deepcopy(current_node.assignment)
75             new_assignment[person] = job
76             new_cost = current_node.cost + cost_matrix[person][
77                 job]
78             new_lower_bound = calculate_lower_bound(cost_matrix,
79                                                         new_assignment,
80                                                         n)
81
82             child = Node(new_assignment, new_cost,
83                             current_node.level + 1, new_lower_bound)
84
85             # 只有下界小于上界才加入队列
86             if child.lower_bound < upper_bound:
87                 heapq.heappush(queue, child)
88
89     return best_solution, upper_bound, nodes_explored, nodes_pruned
90
91 def print_solution(assignment, cost_matrix, total_cost):
92     """打印解决方案"""
93     print("最优分配方案：")
94     print("-" * 30)
95     persons = ['A', 'B', 'C', 'D']
96     jobs = ['Job1', 'Job2', 'Job3', 'Job4']
97
98     for person_idx in sorted(assignment.keys()):

```

```

97     job_idx = assignment[person_idx]
98     cost = cost_matrix[person_idx][job_idx]
99     print(f"{persons[person_idx]} -> {jobs[job_idx]}, 成本: {cost}
100         ")
101
102     print("-" * 30)
103     print(f"总成本: {total_cost}")
104
105 # 主程序
106 if __name__ == "__main__":
107     # 成本矩阵
108     cost_matrix = [
109         [9, 2, 7, 8], # A
110         [6, 4, 3, 7], # B
111         [5, 8, 1, 8], # C
112         [7, 6, 9, 4]  # D
113     ]
114
115     print("任务分配问题 - 分支限界算法")
116     print("=" * 40)
117
118     # 求解
119     solution, min_cost, explored, pruned =
120         branch_and_bound_assignment(cost_matrix)
121
122     # 输出结果
123     print_solution(solution, cost_matrix, min_cost)
124     print(f"\n算法统计:")
125     print(f"探索节点数: {explored}")
126     print(f"剪枝节点数: {pruned}")
127     print(f"剪枝率: {pruned/explored*100:.1f}%")

```

5 算法执行过程

5.1 初始化阶段

- 上界: $UB = +\infty$

- 根节点下界：对每个人选择最小成本工作

$$LB_{\text{root}} = \min(9, 2, 7, 8) + \min(6, 4, 3, 7) + \min(5, 8, 1, 8) + \min(7, 6, 9, 4) \quad (1)$$

$$= 2 + 3 + 1 + 4 = 10 \quad (2)$$

5.2 分支搜索过程

第 1 层：为人员 A 分配工作

- A→Job1: 当前成本 9, 下界 17
- A→Job2: 当前成本 2, 下界 10
- A→Job3: 当前成本 7, 下界 20
- A→Job4: 当前成本 8, 下界 18

选择下界最小的”A→Job2”继续分支。

第 2 层：基于 A→Job2, 为人员 B 分配剩余工作

- A→Job2, B→Job1: 累计成本 8, 下界 13
- A→Job2, B→Job3: 累计成本 5, 下界 14
- A→Job2, B→Job4: 累计成本 9, 下界 17

第 3 层：基于 A→Job2, B→Job1, 为人员 C 分配工作

- A→Job2, B→Job1, C→Job3: 累计成本 9, 总成本 13 (C→Job3 成本 1, D→Job4 成本 4)
- A→Job2, B→Job1, C→Job4: 累计成本 16, 总成本 25 (C→Job4 成本 8, D→Job3 成本 9)

找到完整解, 更新上界: $UB = 13$

5.3 剪枝过程

由于其余所有分支的下界都大于等于 13, 全部被剪枝。

6 问题解答

6.1 原始问题回溯处理方案

任务分配问题可以通过多种回溯处理方案:

6.1.1 方案 1：深度优先回溯

- **策略**：按人员顺序，深度优先搜索所有可能分配
- **优点**：空间复杂度低，实现简单
- **缺点**：可能搜索大量无效分支，时间复杂度高
- **适用**：问题规模较小时

6.1.2 方案 2：分支限界回溯

- **策略**：结合下界估计进行智能剪枝
- **优点**：大幅减少搜索空间，效率高
- **缺点**：需要设计有效的界限函数
- **适用**：中等规模问题的精确求解

6.1.3 方案 3：启发式回溯

- **策略**：使用启发式信息指导搜索顺序
- **优点**：能够更快找到较好解
- **缺点**：不保证全局最优
- **适用**：大规模问题的近似求解

6.2 如何修剪树使搜索空间变小

6.2.1 1. 改进界限函数

当前方法：简单行最小值求和

$$LB = \sum_{i \in \text{未分配}} \min_j c_{ij}$$

改进方法：匈牙利算法计算精确下界

- 对剩余的子问题求解线性规划松弛
- 使用更紧的下界，提高剪枝效率

6.2.2 2. 优化分支策略

当前策略：按人员顺序分支

改进策略：

- **最小成本优先：**优先分配成本最小的人员-工作对
- **最少选择优先：**优先为可选工作最少的人员分配
- **约束最紧优先：**优先处理约束最严格的子问题

6.2.3 3. 启发式初始解

获得好的初始上界可以提早剪枝：

- **贪心算法：**每次选择当前最小成本分配
- **随机算法：**多次随机分配取最优
- **局部搜索：**从随机解开始局部优化

6.2.4 4. 节点选择策略

当前策略：最小下界优先（最优优先搜索）

其他策略：

- **深度优先：**优先扩展最深节点，节省内存
- **广度优先：**逐层搜索，保证解的层次性
- **混合策略：**结合多种策略的优点

6.2.5 5. 对称性剪枝

如果问题具有对称性，可以避免搜索对称的分支：

- 识别等价的人员或工作
- 只搜索代表性分配方案
- 显著减少搜索空间

7 算法性能分析

7.1 时间复杂度

- 最坏情况: $O(n!)$ - 需要搜索所有排列
- 平均情况: 通过剪枝大幅降低, 实际复杂度远小于 $O(n!)$
- 最好情况: $O(n^2)$ - 剪枝非常有效时

7.2 空间复杂度

- 队列存储: $O(b^d)$, 其中 b 是分支因子, d 是深度
- 实际空间: 由于剪枝, 通常远小于理论上界

7.3 算法优势

1. 精确性: 保证找到全局最优解
2. 灵活性: 可适应不同约束条件
3. 可扩展性: 容易加入新的剪枝策略

7.4 算法局限

1. 指数复杂度: 大规模问题计算时间长
2. 内存需求: 可能需要存储大量节点
3. 界限依赖: 效果很大程度依赖界限函数质量

8 最终结果

8.1 最优解

通过分支限界算法, 找到任务分配问题的最优解:

人员	分配工作	成本
A	Job2	2
B	Job1	6
C	Job3	1
D	Job4	4
总成本		13

8.2 算法效果

- **最优性**：找到全局最优解，总成本为 13
- **效率**：通过有效剪枝，避免了大量无效搜索
- **可验证性**：可以验证其他任何分配方案的成本都不小于 13

9 结论

分支限界算法成功解决了 4×4 任务分配问题，展现了以下特点：

1. **系统性**：通过系统的分支搜索，保证解的完整性
2. **智能性**：利用下界估计进行有效剪枝
3. **最优性**：保证找到全局最优解
4. **可扩展性**：算法框架可以应用于更大规模的问题

对于实际应用，建议根据问题规模选择合适的求解方法：小规模问题用精确算法，大规模问题考虑启发式或近似算法。